

On the Shoulders of Feature Preprocessing

Abstract

This report focuses on the design and implementation of a multifunctional classifier to efficiently process different classification tasks, including sentiment analysis, product type classification and helpfulness analysis. A softmax classifier with multinomial logistic regression was chosen as the foundational model. The classifier incorporates a modular design for task switching and flexible extensions of feature engineering by dynamically controlling parameters. This modular system supports unigram and bigram tokenization, one-hot and Term Frequency-Inverse Document Frequency(TF-IDF) encoding, stopword filtering and their combinations, which facilitates an efficient and streamlined experimental process. In the experiments, the system identifies the most suitable model configurations for each task, offering an actionable insight into task-specific optimization on the shoulders of feature preprocessing.

1 Introduction

Efficient text classification is essential for handling tasks such as sentiment analysis, product type classification, and helpfulness analysis, which play a critical role in understanding user feedback, optimizing recommendations, and enhancing review prioritization. This report investigates a modular, parameter-driven classifier based on multinomial logistic regression and softmax function. It has high flexibility and provides a systematic basis for identifying the optimal configuration for each task.

There are two key challenges during experiments: the generalization of the model across multiple tasks and the control of preprocessing steps on classification outcomes. The proposed system addresses these issues through a modular design and a comparative framework. From the perspectives of theory and experiment, the study not only figures out the best-performing model configurations for specific tasks but also sheds light on the individual and synergistic roles of preprocessing strategies in text classification. It can serve as a foundation for future work in modular and extensible classification systems.

2 Data

I trained and evaluated the classifier on the dataset obtained from product reviews on Amazon, a publicly available sources, through the metrics of macroaverage precision and recall.

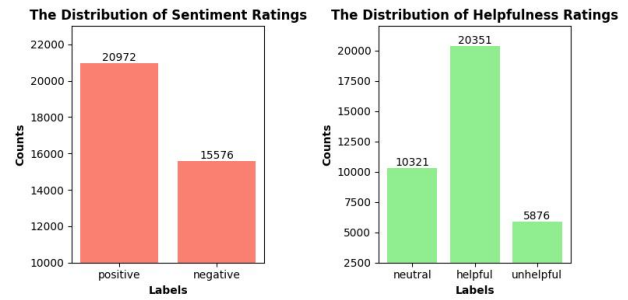


Figure 1: The Distribution of sentiment ratings

Figure 2: The Distribution of sentiment ratings

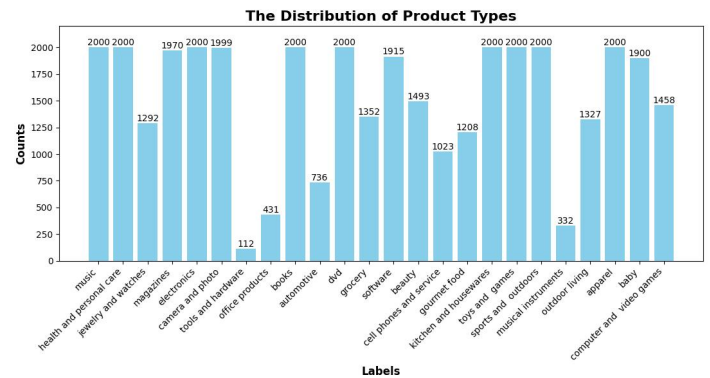


Figure 3: The Distribution of product types

In Figures 1, 2 and 3, there are obvious uneven distributions among the dataset of three tasks. A serious imbalance occurs in the product categories, which may adversely affect the predictions of the classifier, as the model would be biased toward the majority of classes.

4 Experimental Setup

In the experiments, I consider 3 methods of data preprocessing combined with 8 configurations and build a single classifier across three classification tasks. To better isolate the impact of specific factors, I designed a modular system with parameterized management. Each module operates independently but collaboratively, which enables efficient execution and adjusting for diverse configurations with parameters passed via the main() function. The main technical details behind this classifier system can be separated into 3 parts: data preprocessing, model training and model evaluation. There are two types of parameters: 1) To control the desired feature encoding methods; 2) To tune hyperparameters during model training and evaluation.

Related source code is available at:
https://github.com/Mifuxuanan/LELA60331_On-the-shoulders-of-feature-preprocessing.git.

4.1 Basic Theory

In the training phase, the core concept is based on the softmax function and the multiclass cross-entropy loss. After the linear transformation of the feature matrix, the softmax normalizes the output scores into a probability distribution, while the latter is used to evaluate the prediction performance in multi-class classification tasks. In essence, it measures how well the model's predictions align with the actual labels, minimizing the loss as the primary objective of the optimization process. The related formulas are calculate as follows:

$$q_{i,j} = \frac{e^{z_{i,j}}}{\sum_k e^{z_{i,k}}}$$
$$\text{Loss} = -\frac{1}{N} \sum_{j=1} y_{i,j} \cdot \log(\hat{y}_{i,j} + \epsilon)$$

4.2 Feature preprocessing

A standout design of the system is its flexibility to randomly or specifically combine tokenization, feature encoding and stopwords filtering. It results in a comprehensive evaluation of the interaction effects in eight distinct data preprocessing configurations that include:

1. Unigram +One-hot encoding
2. Unigram +TF-IDF
3. Unigram +One-hot encoding+Stopword filtering
4. Unigram +TF-IDF+Stopword filtering
5. Bigram +One-hot encoding
6. Bigram +TF-IDF
7. Bigram +One-hot encoding+Stopword filtering
8. Bigram +TF-IDF + Stopword filtering

First, unigram tokenization focuses on single-token features to get fundamental lexical information, while bigram tokenization captures contextual relationships in the adjacent tokens to provide phrase-level semantics. Second, one-hot encoding converts categorical data into a binary vector representation (unique category with a single “1” and the rest of the indices with “0”s). TF-IDF is a statistical method used to evaluate the importance of a target term in a document relative to a collection of documents (i.e. the corpus). The related formulas are as follows:

$$TF(t, d) = \frac{\text{Number of occurrences of term } t \text{ in document } d}{\text{Total number of terms in document } d}$$

$$IDF(t) = \log\left(\frac{\text{Total number of documents in the corpus}}{\text{Number of documents containing term } t}\right)$$

$$TF - IDF(t, d) = TF(t, d) \times IDF(t)$$

Finally, stopwords filtering can determine whether the removal of common words can reduce noise in the dataset.

4.3 Hyperparameter Tuning

For each of these combination conditions, I turned on a bunch of different optimization-related

hyperparameters and selected a configuration with a batch size of 32 and a learning rate of 0.001 for 200 epochs, which led to the most balanced performance of the classifier across several tasks. I also changed the regular expression during the step of word segmentation, such as “\w+|[\^\w\s]” in my initial few experiments to separate words from punctuation in the raw dataset. However, considering the need to control multiple variables and to avoid conflicts with observing the effect of bigram word segmentation, I decided to retain the original pattern “[^]+” of dividing words only by spaces.

I trained the classifier on the first 2500 tokens from the most frequent ones as well as 5000 and 10000 tokens as comparative experiments. However, the latter choice only had slight improvement on every trial. Given the efficiency, I finally focus on the condition of the top 2500 tokens. In the case of convergence, not every training process achieved perfect performance after completion of all epochs. For the unsatisfactory results, increased epochs and different learning rates still do not improve much. Because of the equipment and time limitations, more controlled studies are left for further exploration.

5 Results

In the experiment, task complexity and the strategies of feature selection had a major impact on model performance. The important details of all evaluation results on different combinations of three tasks and eight methods have been summarized in Tables/Figures 1, 2 and 3.

5.1 Main conclusions

A. Simpler task achieved higher precision and recall. As the model can easily learn the class boundaries, sentiment analysis got macro average precision(0.675) and recall(0.713) on average. In contrast, product types with 24 classes got only 0.343 and 0.312 respectively. This reflects the difficulty of multi-class classification with unbalanced data distribution and sparse features.

B. Unigram tokenization outperformed bigram tokenization across all tasks. Bigram tokenization, which was expected to capture sequential dependencies, often failed to converge without preprocessing such as TF-IDF or stopwords filtering. The precision and recall of only bigram tokenization in Table 1,2 and 3 all show zeros in most cases. In helpfulness classification, bigram alone resulted in 0.054 macro precision and 0.333 macro recall, while “Unigram+TF-IDF” significantly improved macro precision to 0.596. These results highlight the limitations of bigram features in handling high-dimensional and sparse datasets.

Sentiment	Macro Precision	Macro Recall	Precision	Recall
Unigram	0.820	0.825	[0.871, 0.768]	[0.821, 0.829]
Unigram+TF-IDF	0.812	0.779	[0.784, 0.841]	[0.914, 0.643]
Unigram-stop words	0.808	0.81	[0.847, 0.769]	[0.833, 0.787]
Unigram+Both	0.829	0.82	[0.837, 0.822]	[0.884, 0.756]
Bigram	0.293	0.5	[0.586, 0.000]	[1.000, 0.000]
Bigram+TF-IDF	0.793	0.742	[0.748, 0.837]	[0.922, 0.561]
Bigram-stop words	0.293	0.5	[0.586, 0.000]	[1.000, 0.000]
Bigram+Both	0.749	0.735	[0.762, 0.735]	[0.839, 0.630]
Average_Sentiment	0.675	0.713	/	/

Table 1: Evaluation of sentiment classification

Helpfulness	Macro Precision	Macro Recall	Precision	Recall
Unigram	0.448	0.449	[0.368, 0.331, 0.647]	[0.356, 0.497, 0.494]
Unigram+TF-IDF	0.596	0.35	[0.625, 0.612, 0.551]	[0.008, 0.056, 0.984]
Unigram-stop words*	0.455	0.433	[0.383, 0.393, 0.587]	[0.306, 0.27, 0.723]
Unigram+Both	0.585	0.379	[0.65, 0.541, 0.563]	[0.066, 0.122, 0.95]
Bigram	0.054	0.333	[0.162, 0.0, 0.0]	[1.0, 0.0, 0.0]
Bigram+TF-IDF	0.498	0.343	[0.48, 0.467, 0.548]	[0.02, 0.026, 0.984]
Bigram-stop words	0.054	0.333	[0.162, 0.0, 0.0]	[1.0, 0.0, 0.0]
Bigram+Both	0.389	0.378	[0.245, 0.351, 0.572]	[0.228, 0.119, 0.787]
Average_Helpfulness	0.385	0.375	/	/

Table 3: Evaluation of helpfulness classification

Product Types	Macro Precision	Macro Recall	Precision (example)	Recall (example)
Unigram	0.559	0.551	[0.396, 0.549, 0.922, ...]	[0.424, 0.632, 0.910, ...]
Unigram+TF-IDF	0.574	0.434	[0.357, 0.301, 0.743, ...]	[0.320, 0.713, 0.802, ...]
Unigram-stop words	0.583	0.565	[0.398, 0.601, 0.876, ...]	[0.517, 0.619, 0.834, ...]
Unigram+Both	0.64	0.505	[0.374, 0.417, 0.866, ...]	[0.468, 0.673, 0.829, ...]
Bigram	0.002	0.042	[0.056, 0.000, 0.000, ...]	[1.000, 0.000, 0.000, ...]
Bigram+TF-IDF	0.249	0.236	[0.138, 0.208, 0.462, ...]	[0.143, 0.337, 0.524, ...]
Bigram-stop words	0.002	0.042	[0.056, 0.000, 0.000, ...]	[1.000, 0.000, 0.000, ...]
Bigram+Both	0.136	0.123	[0.056, 0.112, 0.250, ...]	[0.177, 0.104, 0.214, ...]
Average_Types	0.343	0.312	/	/

Table 2: Evaluation of product types classification

C. Preprocessing methods (TF-IDF and stopword filtering) improved the classifier's performance. While these two methods indeed improved unigram-based models, they did not resolve issues such as low recall and class imbalances in multi-class tasks. More methods like data augmentation, advanced feature engineering, and optimization of hyperparameters should be considered to better capture nuanced dependencies in context.

5.2 Role of Weights

The classification decisions made by the classifier are heavily influenced by the importance of features. The weight matrix offers a deeper understanding and interpretation of these processes. High-weight features often present strong correlations with specific classes, such as emotional tokens or product-specific keywords. In sentiment analysis, the preprocessing method "Unigram+Both" achieved the best performance both on macro precision(0.829) and macro recall (0.820).

The related weight matrix sample (the first 5 rows) is as follows:

```
[[[-5.223, 6.015]
 [1.075, 0.307]
 [0.707, 0.016]
 [-0.566, 1.525]
 [1.241, -0.983]]]
```

The first line is a good example of a highly distinguishing feature. With the first number -5.223, the model tends to identify this token as higher likelihood of belonging to another class. At the same time, the next positive number 6.015 further supports this assumption, which has a strong positive contribution to the prediction of the current class. The rest of the weights in this sample also present a clear distinction in pairs. This weight matrix indicates that the model successfully captured a keyword to recognize the sentiment classification. In contrast, when it comes to product types, the classifier with "Bigram" faced the problem of gradient vanishing. Its precision and recall were close to zero as well as a terrible weight matrix with almost no difference among 24 dimensions.

6 Discussion

The modular framework significantly reduces the time for model tuning and improves the efficiency and reproducibility of experiments. Parameterized management enables precise control over different key variables to get the target experimental result. Another major feature of this report is its systematic design of experiments. It allows the nuanced exploration of the model's ability and limitations on different classification tasks sharing the same preprocessing step as well as within different feature engineering methods under the same task. This architecture has high generalizability and is not only suitable for multi-task and multi-class classification studies but also lays the foundation for adapting different methods of feature engineering to other text-based

tasks.

6.1 Limitations of Bigram Tokenization

With token pairs, the model was supposed to better understand sequential dependencies and have more potential for tasks requiring more nuanced understanding, such as product categorization. However, the experimental results did not align with my initial hypothesis. The poor performance of the model using bigram tokenization and “bigram+stopword filtering” is observed on every task, especially the product types classification with near zero precision and recall.

It is hard to deny that bigram tokenization has a much higher dimensional feature than that of Unigram, which introduces the challenge of feature sparsity. For a vocabulary of 2500 most frequent tokens, the number of potential features in Bigram is 6,250,000 (2500^2). It is hard for the model to learn the weights of many bigram combinations that rarely or never appear in the training set, particularly in datasets with limited size. Rather than a large number of low-frequency tokens, only a few high-frequency tokens contribute to the bigram tokenization, which increases difficulties in identifying specific features.

Moreover, the lengths of each review can not provide enough samples to cover all needed features, let alone the sensitivity of bigram tokenization to noise like typos and punctuation. The unbalanced distribution in the dataset, such as “tools and hardware” only occurs 112 times in product types, further deteriorates the learning ability of the model on low-frequency classes. Lastly, bigram is unable to capture long-distance dependencies in context.

6.2 Impact of TF-IDF and Stopword Filtering

By enhancing feature relevance, the options of “TF-IDF” and “Stopword Filtering” both strengthen the classifier’s performance in most cases, especially their combination achieves a best result of macro precision and macro recall in two tasks.

On one hand, TF-IDF uses frequency-based weighting to make informative tokens more important in classification decisions. For example, although some common words like “review” or “product” are more common in the dataset, only domain-specific words like “useful” or “durable” have higher weights in TF-IDF representations. In the helpfulness classification, unigram tokenization with TF-IDF increased macro average precision to 0.596 compared to that without TF-IDF (0.448), which indicates that this method focuses on

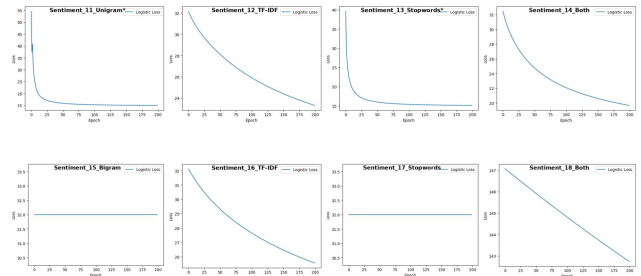


Figure 4: loss plot of sentiment ratings

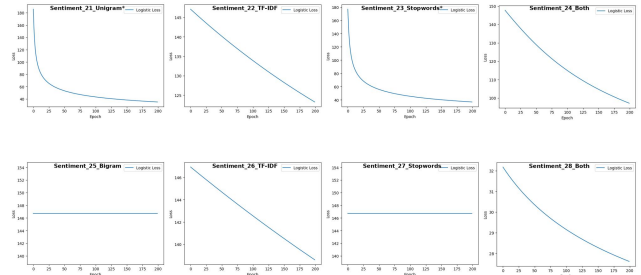


Figure 5: loss plot of product types

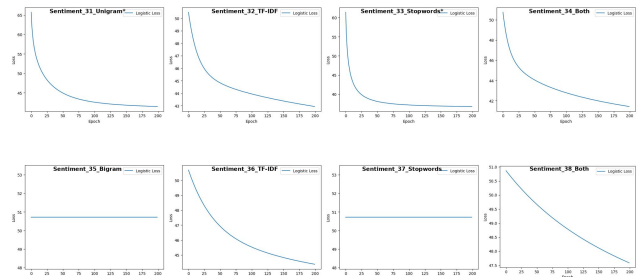


Figure 6: loss plot of helpfulness

Task-relevant terms strongly correlate with helpfulness ratings. On the other hand, with a similar logic, stopwords filtering can also improve efficiency for training models by removing some high-frequency but meaningless tokens from the dataset. For sentiment analysis, it helps the model prioritize words with strong positive or negative meanings like “excellent” or “poor” over neutral connectors.

Although these two methods provide complementary benefits to the classifier, their impact is still constrained by some fundamental limitations that also hinder the above bigram tokenization. TF-IDF struggles to assign meaningful weights to underrepresented categories in the dataset, which leads to a limited improvement in macro recall of product types task, rising only from 0.551(Unigram) to 0.565(Unigram+TF-IDF). Similarly, stopwords filtering can only marginally reduce the noise in the reviews, as observed in the helpfulness classification, where this method increased macro precision slightly from 0.448(Unigram) to 0.455(Unigram+Stopword filtering).

6.3 Future Directions

To improve the performance of this classification system, several potential methods are left for further experiments. First, a grid search procedure can systematically explore more hyperparameters and create better configurations for optimization. Second, introducing deep learning models will bring non-linear feature interactions. The functions of such models undoubtedly outperform logistic regression for complex NLP tasks, especially the self-attention mechanism of Transformers. Third, adjusting more details during data preprocessing, such as adding samples of rare classes or using resampling methods. Word embedding like word2vec also would be a better choice to capture semantic information for tasks like helpfulness analysis. Further experiments can also focus on improving the dataset by adding samples of rare categories or using resampling methods. Finally, potential negative impacts like biased predictions for minority classes may lead to unfair treatment of certain product reviews. To avoid the related risks, fairness adjustments and transparency evaluations should be incorporated into real-world applications. Overall, these directions provide different pathways for extending the ability of this classifier across multi-task, multi-lingual and multi-model research.

7 Conclusion

Standing on the shoulders of feature engineering, this study designed a modular, multi-task classifier based on multinomial logistic regression and demonstrated the impact of different preprocessing methods on the model's performance across three classification tasks. Combined with theoretical and experimental verification, "Unigram tokenization with TF-IDF and stopword filtering" is the optimal configuration. Future work will explore the ability and limitations of different classifiers through advanced feature engineering and model architectures.

Reference

Jurafsky, D. and J. H. Martin (2023). Speech and language processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition. 3rd Edition. Free online at <https://web.stanford.edu/~jurafsky/slp3/>

sebleier, NLTK's list of english stopwords. Free online at <https://gist.github.com/554280.git>